04 — PHP for the Web

All examples in this module are web pages. Run them with `php -S localhost:8000` from the folder containing them.

HTTP in 1 minute

Every page load is a **request** from the browser and a **response** from the server. HTTP is **stateless**: the server doesn't remember the previous request. Forms, sessions and cookies exist to work around that.

- **GET**: asks for a page. Parameters are visible in the URL: `search.php?q=php&page=2`. Use it for reading (search, filters, pagination).
- **POST**: sends data in the request body. Use it for anything that **changes** something (create an account, send a message, upload a file).

PHP fills these superglobal arrays for you:

Variable	Contains
`$_GET`	URL parameters
`$_POST`	form fields sent with POST
`$_FILES`	uploaded files
`$_SESSION`	session data (after `session_start()`)
`$_COOKIE`	cookies sent by the browser
`$_SERVER`	request info: `$_SERVER['REQUEST_METHOD']`, `$_SERVER['REQUEST_URI']`…

Sending headers from PHP: `header('Location: thanks.php'); exit;` (redirect). Headers must be sent **before any output** (even a space or a blank line before `<?php`), otherwise: "*Cannot modify header information - headers already sent*".

Forms

```
<form method="post" action="contact.php">
    <input type="text" name="name">
    <input type="email" name="email">
    <textarea name="message"></textarea>
    <button type="submit">Send</button>
</form>
```

The `name` attribute becomes the key: `$_POST['name']`, `$_POST['email']`…

Rule #1: never trust user input

Anything coming from the browser can be modified by the user (DevTools, curl…). HTML validation (`required`, `type="email"`) is for comfort, **not security**. Always validate on the server.

```
$errors = [];
$name  = trim($_POST['name'] ?? '');
$email = trim($_POST['email'] ?? '');

if ($name === '') {
    $errors['name'] = 'Name is required.';
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Invalid email.';
}
```

Rule #2: escape everything you display

If you display user input as is, a user can inject HTML or JavaScript into your page: that's an **XSS** attack.

```
echo $_GET['q'];                                  // DANGEROUS
echo htmlspecialchars($_GET['q'] ?? '');          // safe: < > " ' & become entities
```

Tip: create a tiny helper and use it everywhere.

```php
function e(?string $value): string
{
    return htmlspecialchars($value ?? '', ENT_QUOTES, 'UTF-8');
}
```

The usual form pattern

One file both **displays** and **processes** the form:

```php
<?php
$errors = [];
$name = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    if ($name === '') {
        $errors['name'] = 'Required';
    }
    if (!$errors) {
        // ... save, send mail, etc.
        header('Location: thanks.php');   // Post/Redirect/Get: avoids re-sending on refresh
        exit;
    }
}
?>
<form method="post">
    <input name="name" value="<?= e($name) ?>">   <!-- keep what the user typed -->
    <?= isset($errors['name']) ? '<p>' . e($errors['name']) . '</p>' : '' ?>
    <button>Send</button>
</form>
```

Sessions

A session stores data **on the server**, linked to a visitor via a cookie (`PHPSESSID`). Typical use: the logged-in user, a shopping cart, flash messages.

```php
session_start();                        // at the top of EVERY page using the session, before any output

$_SESSION['user'] = 'alice';            // write
echo $_SESSION['user'] ?? 'guest';      // read
unset($_SESSION['cart']);               // remove one key

// Logout
$_SESSION = [];
session_destroy();
```

After a successful login, call `session_regenerate_id(true);` to prevent session fixation attacks.

Passwords

Never store a password in clear. PHP does the hard part:

```php
$hash = password_hash('secret123', PASSWORD_DEFAULT);   // store this in the database
password_verify('secret123', $hash);                    // true
```

Cookies

A cookie is stored **in the browser** and sent back with every request. Small, not secret, the user can read and modify it. Use it for preferences (language, theme), not for sensitive data.

```php
setcookie('theme', 'dark', [
    'expires'  => time() + 60 * 60 * 24 * 30,   // 30 days
    'path'     => '/',
    'httponly' => true,       // not readable from JavaScript
    'samesite' => 'Lax',
]);

$theme = $_COOKIE['theme'] ?? 'light';
```

Like `header()`, `setcookie()` must be called before any output. The new value appears in `$_COOKIE` on the **next** request

only.

File uploads

The form needs `method="post"` **and** `enctype="multipart/form-data"`:

```
<form method="post" enctype="multipart/form-data">
    <input type="file" name="avatar">
    <button>Upload</button>
</form>
```

```
$file = $_FILES['avatar'];
// $file['name']      original name (from the user: don't trust it)
// $file['tmp_name']  temporary path on the server
// $file['size']      size in bytes
// $file['error']     UPLOAD_ERR_OK (0) if everything went fine
```

Security checklist:

1. Check `$file['error'] === UPLOAD_ERR_OK`.
2. Check the size.
3. Check the **real** type with `finfo` (the extension and `$file['type']` come from the user and can lie).
4. Generate your own file name (never reuse the user's name: `../../index.php`...).
5. Store outside your code folder if possible, and never allow executing uploaded `.php` files.
6. Move it with `move_uploaded_file()`.

```
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($file['tmp_name']);
$allowed = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp'];

if (!isset($allowed[$mime])) { /* error */ }

$newName = bin2hex(random_bytes(8)) . '.' . $allowed[$mime];
move_uploaded_file($file['tmp_name'], __DIR__ . '/uploads/' . $newName);
```

Server limits are set in `php.ini`: `upload_max_filesize` and `post_max_size`.

Working with files in general: `file_exists`, `is_dir`, `mkdir`, `file_get_contents`, `file_put_contents`, `unlink` (delete), `scandir`.

Debugging

In development, see all errors:

```
ini_set('display_errors', '1');
error_reporting(E_ALL);
```

Your tools, from simplest to most powerful:

Tool	Use
`var_dump($x); exit;`	inspect a value at a precise point
`error_log('msg ' . print_r($x, true));`	log without breaking the page (appears in the `php -S` terminal)
The `php -S` terminal	shows each request, its status code, and logged errors
Browser DevTools (F12) → Network	see the requests, sent form data, cookies, status codes
`php -l file.php`	syntax check without running
Xdebug + VS Code (*PHP Debug* extension)	breakpoints, step by step, inspect variables

Reading an error message: **type** (Warning, Fatal error, Parse error), **message**, **file** and **line**. A *Parse error* is often caused by the line **before** the one shown (missing `;` or `}`).

For performance: `microtime(true)` before/after a block gives its duration; Xdebug also has a profiler.

Now do the exercises.

04 — Exercises

Run `php -S localhost:8000` in your working folder and open the pages in the browser.

4.1 — Search page (GET)

Create `search.php` with a search field (`name="q"`) and `method="get"`.

- When a search is submitted, display `You searched for: ...` under the form.
- Filter this list and display only the matching items (case-insensitive, `stripos` or `str_contains` + `strtolower`):

```php
$languages = ['PHP', 'Python', 'Java', 'JavaScript', 'Go', 'Rust', 'Ruby'];
```

- The search field must keep the searched text after submitting.
- Test the security: search for `<script>alert('hacked')</script>`. Nothing should pop up.

4.2 — Contact form (POST)

Create `contact.php` with fields: name, email, subject (a `<select>` with 3 options), message.

Server-side validation:

- name: required
- email: required and valid (`filter_var`)
- subject: must be one of the 3 allowed values (don't trust the `<select>`)
- message: at least 10 characters

If there are errors, show each error next to its field and keep the values the user typed. If everything is valid, redirect to `thanks.php` (Post/Redirect/Get), which displays a confirmation.

Bonus: pass the sender's name to `thanks.php` through the session, and display it once ("flash message").

4.3 — Login with sessions

Create 3 pages:

- `login.php`: a login form. Valid credentials: `admin` / `secret123`. Store the **hash** of the password in your code (generate it once with `password_hash` in a separate script), and check with `password_verify`.
- `dashboard.php`: only accessible when logged in. Displays `Welcome, admin` and a logout link. If not logged in, redirect to `login.php`.
- `logout.php`: destroys the session and redirects to `login.php`.

Check in DevTools (Application → Cookies) that a `PHPSESSID` cookie appears.

4.4 — Theme preference (cookie)

Create `theme.php` with two links: `?theme=light` and `?theme=dark`.

- Clicking a link saves the choice in a cookie for 30 days, then redirects to `theme.php` (without the parameter).
- The page uses the cookie value to set its colors (white or dark background).
- Only accept `light` or `dark` as values.
- Close the browser, come back: the theme is remembered.

4.5 — Avatar upload

Create `upload.php`:

- A form to upload an image.
- Accept only JPEG, PNG or WebP (checked with `finfo`), max 2 MB.
- Save it into an `uploads/` folder (create it with `mkdir` if it doesn't exist) under a random name.
- Display the list of uploaded images as thumbnails (`scandir` or `glob`).
- Test the security: rename a `.txt` file to `.jpg` and try to upload it. It must be rejected.

4.6 — Bug hunt

Copy `buggy.php` into your folder and open it in the browser. It's supposed to display a list of users with their age and the average age.

It contains **5 bugs**. Use error messages, `var_dump` and `php -l` to find and fix them all.